

SLE-2: Profiling Report

Course	02AML204 – Introduction to Artificial Intelligence
PRN	26UAM307
Name	Shravani Sadashiv Bramhadande
Division	A
Date	22-09-2026
GitHub Link	https://github.com/shravani9103/AI-SLE2

1. Algorithms / Versions Profiled

- Algorithm A: Breadth-First Search (BFS)
- Algorithm B: Depth-First Search (DFS)
- Problem used: Small graph search problem
- Start node: A

The purpose of the search was to reach the goal node AD starting from node A using two different uninformed search strategies.

- Goal node: AD

2. Profiling Method

- Tool used: Python time it module and manual node counting.
- How I measured: Both BFS and DFS were executed on the same graph. The number of nodes expanded by each algorithm was counted. Execution time was measured using the Python timeit module. Each algorithm was tested using 1,000 profiling executions, and the average time was calculated.
- Number of runs: 1,000 executions

The same graph and start/goal nodes were used for both algorithms so that the comparison remained fair. The experiment focused on two practical measures: execution time and the number of nodes expanded.

For each algorithm, the program was executed repeatedly using the same graph and the same start and goal nodes. The time it module recorded the execution time across repeated runs, while the program separately counted the nodes expanded before reaching the goal. The repeated timing measurements were averaged to obtain a representative result for the experiment.

3. Results

Metric	BFS	DFS
Average Time	0.011028 ms	0.013273 ms
Nodes Expanded	30	30
Path Found	A → C → G → O → AD	A → C → G → O → AD

The timing values represent the average execution time calculated over 1,000 profiling executions. Repeated executions were used so that the timing difference could be measured more consistently than a single graph-search execution.

Using the same input for both algorithms makes the comparison more meaningful because the difference is based on the search strategy rather than a change in the problem. The measured values show that both algorithms expanded the same number of nodes for this graph.

Short Observation: From the profiling results, BFS took less measured time than DFS for the selected graph. Both BFS and DFS expanded 30 nodes. Therefore, based on the measured results for this particular graph, BFS showed lower execution time while both algorithms had equal node expansions.

4. Justification & Analysis

For the selected graph, BFS showed lower measured execution time than DFS. The average time was 0.011028 ms for BFS compared with 0.013273 ms for DFS. Both algorithms expanded 30 nodes. Therefore, for this particular test case, BFS required less measured time while the amount of node expansion was the same. The result is based on the actual profiling output rather than theory alone.

These results are specific to the selected graph and the chosen start and goal nodes. Changing the graph structure or placing the goal at a different location can change how many nodes each search explores and can therefore change the measured execution time. This shows why empirical profiling is useful in addition to theoretical analysis.

The profiling results also illustrate that theoretical properties do not always translate into the same measured result for every small test case. The location of the goal and the order in which neighboring nodes are explored affect the number of nodes visited. In this experiment, both algorithms expanded 30 nodes, but their measured execution times were slightly different.

5. AI Contribution Note

- AI tools used: ChatGPT.
- What AI helped with: Understanding the SLE-2 requirements, preparing the BFS and DFS profiling code, explaining the profiling method, and organizing the report.
- What I did myself: Run the program, performed the profiling experiments, verified the results, and prepared the final analysis and report.

6. Conclusion

This profiling exercise helped me understand how the practical performance of search algorithms can be measured. I compared BFS and DFS on the same graph using execution time and nodes expanded. The experiment showed that the measured performance depends on the search strategy and the structure of the problem. For the selected graph, BFS had lower measured time while both algorithms had the same number of node expansions. I learned that profiling provides actual performance data that can be used to compare algorithms and support observations with evidence.

OUTPUT:

```
Problems 2 Output Debug Console Terminal Ports

PS C:\Users\SHRAVANI\Downloads\AI-Agent-SLE1-main> python sle2.py
PS C:\Users\SHRAVANI\Downloads\AI-Agent-SLE1-main> python sle2.py

=====
SLE-2: BFS vs DFS
=====

Problem:
Start Node : A
Goal Node : AD

Number of Profiling Runs: 1000

----- BFS -----
Average Time: 0.011028 ms
Nodes Expanded: 30
Path Found: A -> C -> G -> O -> AD

----- DFS -----
Average Time: 0.013273 ms
Nodes Expanded: 30
Path Found: A -> C -> G -> O -> AD

=====
COMPARISON
=====
Lower execution time: BFS
Both expanded the same number of nodes.
PS C:\Users\SHRAVANI\Downloads\AI-Agent-SLE1-main> 
```